

PYTHON DECORATORS

FARAH YOUNAS

Table of Contents

1. What is a Decorator? 3
2. Basic Decorator 3
3. What Does @decorator Mean? 4
4. Functions as Arguments 4
5. Functions Inside Functions 4
6. Returning a Function 4
7. Decorators with Arguments 5
8. Why Use *args and **kwargs? 5
9. Real-World Uses of Decorators 5
10. Example: Timing a Function 6
11. Common Built-in Python Decorators 6
12. Key Concept to Remember 8
13. Quick Practice Activity 8

1. What is a Decorator?

A decorator is a function that adds extra functionality to another function without changing the original function's code.

Existing function + decorator = enhanced function

2. Basic Decorator

```
# 1. Define the decorator function
# This defines a function called decorator_function,
# It takes one argument: original_function. This argument will be the function that we want to decorate.
# So conceptually decorator_function(hello)

# 2. Define the wrapper function
# The wrapper function will contain the extra functionality that we want to add around the original function.
'''
Before original function
    ↓
 original function
    ↓
After original function
'''
def decorator_function(original_function): # 1. Define the decorator function
  def wrapper(): # 2. Define the wrapper function
    print("Starting...")
    original_function() # 3. Call the original function, calls whatever function was passed into
   decorator_function.
    print("Finished...")
  return wrapper # 4. Return the wrapper

@decorator_function # decorator syntax equivalent to hello = decorator_function(hello)
def hello():
    print("Hello!")

hello()
```

```
def decorator(func):
    def wrapper():
        print("Something is happening before the function is called.")
        func()
        print("Something is happening after the function is called.")
    return wrapper

def say_whee():
    print("Whee!")

say_whee = decorator(say_whee)
say_whee()
```

3. What Does @decorator Mean?

The @decorator syntax is shorthand for replacing the original function with the decorated version.

```
@decorator_function
def hello():
    print("Hello!")
```

is equivalent to:

```
def hello():
    print("Hello!")

hello = decorator_function(hello)
```

4. Functions as Arguments

Python allows a function to be passed as an argument to another function.

```
def greet():
    print("Hello")

def call_function(function):
    function()

call_function(greet)
```

5. Functions Inside Functions

```
def outer():
    def inner():
        print("Inside inner function")
    inner()
outer()
```

6. Returning a Function

```
def outer():
    def inner():
        print("Inside inner function")
    return inner
result = outer()
result()
```

7. Decorators with Arguments

If the decorated function accepts arguments, the wrapper can use *args and **kwargs.

```
def decorator_function(original_function):
    def wrapper(*args, **kwargs):
        print("Function is starting")
        result = original_function(*args, **kwargs)
        print("Function is finished")
        return result
    return wrapper

@decorator_function
def add(a, b):
    return a + b

result = add(10, 20)
print(result)
```

Output:

Function is starting

Function is finished

30

8. Why Use *args and **kwargs?

*args allows the decorator to accept positional arguments, while **kwargs allows it to accept keyword arguments. This makes the decorator flexible enough to work with many different functions.

```
@decorator_function
def hello(name):
    print("Hello", name)
```

or

```
@decorator_function
def calculate(a, b, c):
    return a + b + c
```

The same decorator can handle both.

```
hello("Alex")
calculate(10, 20, 30)
```

9. Real-World Uses of Decorators

- Logging
- Authentication
- Access control
- Timing functions
- Caching
- Validation

- Error handling
- Permission checking

10. Example: Timing a Function

```
import time

def timer(function):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = function(*args, **kwargs)
        end = time.time()
        print("Time taken:", end - start)
        return result
    return wrapper

@timer
def calculate():
    for i in range(1000000):
        pass

calculate()
```

11. Common Built-in Python Decorators

@staticmethod – defines a method that does not require an instance or class reference.

```
class Student:
    @staticmethod
    def university():
        print("Yoobee")
```

@classmethod – defines a method that receives the class (cls) as its first argument.

```
class Student:
  university = "Yoobee"
    @classmethod
    def show_university(cls):
        print(cls.university)
```

@property – allows a method to be accessed like an attribute.

```
class Student:
    def __init__(self, name):
        self._name = name

    @property
    def name(self):
        return self._name

student = Student("Alex")
print(student.name)
```

```
class Decorator(object):
    def __init__(self, func):
        self.function=func

    def __call__(self, a, b):
        result = self.function(a ,b)
        return result ** 2


def add(a, b):
    return a+b

add = Decorator(add)
add(1,2) # add.__call__()
```

```
class Decorator(object):
    def __init__(self, func):
        self.function=func

    def __call__(self, a, b):
        result = self.function(a ,b)
        return result ** 2

@Decorator
def add(a, b):
    return a+b

print(add(2,3))
```

```
class Decorator(object):
    def __init__(self, func):
        self.function=func

    def __call__(self, *args):
        try:
            if any([isinstance(i, str) for i in args]):
                raise TypeError("Cannot pass string as argument")
            else:
                return self.function(*args)
        except Exception as obj:
            print(obj)

def add(*args):
    sum1=0
    for num in args:
        sum1=sum1+num
    return sum1


add=Decorator(add)
print(add(10, 20, 30))
add(10, '20', 30) # add.__call__(10, '20', 30)
```

```
class Decorator(object):
    def __init__(self, func):
        self.function=func

    def __call__(self, *args):
        try:
            if any([isinstance(i, str) for i in args]):
                raise TypeError("Cannot pass string as argument")
            else:
                return self.function(*args)
        except Exception as obj:
            print(obj)

@Decorator
def add(*args):
    sum1=0
    for num in args:
        sum1=sum1+num
    return sum1

print(add(10,20,30))
print(add(10, '20', 30))
```

12. Key Concept to Remember

Decorators wrap functions.

Decorator → takes a function → creates a wrapper → adds extra functionality → returns the wrapper

13. Quick Practice Activity

Create a decorator called @welcome that prints 'Welcome to Python!' before executing a function. Then apply it to a function called student().